PROJECT REPORT

CabAnalytics

Large-Scale Taxi Data Processing and Predictive Analytics

Submitted by

Alwin Wilson | Reg. No: 12310500

Course Code: INT315

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab — 144411

TABLE OF CONTENTS

1 Introduction

1.1 Background of the Study
1.2 Problem Statement
1.3 Objectives
1.4 Scope and overview

2 Problem Analysis

2.1 Problem Statement
2.2 Project Rationale and Scope
2.3 Feasibility study

3 Dataset Description

3.1 Source
3.2 Coverage
3.3 Schema and Data volume

4 Tools and Technologies

5 System Design and Methodology

5.1 Architecture
5.2 Data pipeline
5.3 Training Workflow

6 Implementation

6.1 Implementation of the Project

7 Testing and Validation

7.1 Data quality checks
7.2 Model evaluation procedure

8 Results and Analysis

8.1 Dataset retention
8.2 Demand patterns
8.3 Fare-Distance relationships
8.4 Location analysis and model comparison

9 Limitations and Future Scope

9.1 Local execution constraints
9.2 January only coverage
9.3 Data quality limitations and possible extensions

10 Conclusion

1. INTRODUCTION

1.1 Background of the Study

The increasing availability of large datasets has created opportunities to extract meaningful insights from information generated by transportation systems, financial services, online platforms, and other data-intensive applications. However, analysing such datasets can be challenging when the volume of data increases beyond what can be processed efficiently using conventional approaches. Distributed data-processing frameworks address this challenge by providing tools for data ingestion, transformation, aggregation, and analysis at scale.

Apache Spark is a distributed data-processing engine designed to support large-scale data analytics. Through its Python interface, PySpark, it allows developers to process structured datasets using DataFrame operations and Spark SQL, while also providing machine-learning functionality through MLlib. These capabilities make Spark suitable for projects that combine data preparation, exploratory analysis, and predictive modelling within a common processing workflow.

The New York City Yellow Taxi trip records provide a practical dataset for studying transportation patterns and fare-related behaviour. The records contain information such as pickup and drop-off timestamps, trip distance, passenger count, taxi-zone identifiers, and fare amounts. Analysing these attributes can reveal patterns in trip demand across different hours and days, differences in fares across trip-distance categories, and frequently used pickup and drop-off zones.

This project, titled “Scalable Taxi Trip Analytics and Fare Prediction Using Apache PySpark,” uses January 2025 Yellow Taxi trip records to develop an end-to-end data-processing and analysis pipeline. The workflow includes data acquisition, conversion to a typed Parquet dataset, data cleaning, feature engineering, analytical queries using Spark SQL, and fare prediction using machine-learning models. Linear Regression and Random Forest are evaluated using standard regression metrics to compare their performance on the prepared dataset.

The project demonstrates how PySpark can be used to organise a large transportation dataset into a reproducible analytical workflow, combining descriptive analysis with predictive modelling. It also provides practical experience with data-quality checks, columnar storage, feature preparation, SQL-based aggregation, and model evaluation.

1.2 Problem Statement

Large transportation datasets contain valuable information about travel demand, trip characteristics, and fare patterns. However, extracting reliable insights from such data requires more than simply loading and analysing the raw records. The dataset must be processed systematically to handle missing values, invalid measurements, inconsistent records, and extreme values that may distort statistical summaries and predictive models.

The January 2025 New York City Yellow Taxi trip dataset contains millions of records, making it a suitable case study for implementing a structured data-processing workflow using Apache PySpark. Analysing this volume of data presents practical challenges in data ingestion, storage, transformation, and aggregation. In addition, raw fare and trip measurements may contain anomalous values that require investigation and appropriate treatment before meaningful analysis can be performed.

Another challenge is understanding the factors associated with taxi fares and identifying patterns in trip demand. Trip distance, pickup time, day of the week, and pickup and drop-off locations provide useful attributes for examining variations in fare amounts. Without a consistent analytical pipeline, it becomes difficult to combine data preparation, descriptive analysis, and predictive modelling into a reproducible process.

Therefore, the problem addressed by this project is to develop an end-to-end PySpark-based workflow that can process a large volume of taxi-trip records, prepare the data for analysis, identify meaningful transportation and fare patterns, and evaluate machine-learning models for predicting trip fare amounts.

The project focuses on demonstrating the practical application of Apache Spark for large-scale data processing and analytics in a local computing environment. It does not attempt to implement a production-grade, multi-node transportation analytics platform or guarantee exact fare predictions for every individual trip.

1.3 Objectives

The primary objective of this project is to develop a scalable data-processing and analytical workflow using Apache PySpark to examine New York City Yellow Taxi trip records and evaluate the feasibility of predicting taxi fares using machine-learning techniques.

The specific objectives of the project are as follows:

- Data Acquisition and Ingestion: Obtain the January 2025 NYC Yellow Taxi trip records and organize the raw data into a format suitable for further processing.
- Data Cleaning and Quality Assessment: Identify and handle missing values, invalid trip measurements, and anomalous records using defined data-quality rules to improve the reliability of subsequent analysis.
- Data Transformation and Feature Engineering: Convert raw records into a structured dataset, derive relevant temporal features such as pickup hour and day of the week, and store the processed data in Parquet format.

- Exploratory Data Analysis Using Spark SQL: Analyze trip demand by hour and day, investigate the relationship between trip distance and fare amount, identify frequently used pickup and drop-off locations, and examine passenger-count patterns.
- Machine-Learning-Based Fare Prediction: Develop baseline fare-prediction models using Linear Regression and Random Forest through PySpark's machine-learning capabilities.
- Model Evaluation and Comparison: Evaluate both models using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination ((R^2)) to compare their predictive performance on a held-out test dataset.
- End-to-End Pipeline Integration: Organize the individual processing stages into a reproducible workflow that demonstrates the application of Apache PySpark to large-scale data preparation, analytics, and predictive modelling.

The successful completion of these objectives is intended to demonstrate practical knowledge of big-data processing concepts, Spark SQL, feature engineering, and machine-learning evaluation using a real-world transportation dataset.

1.4 Scope of the Project

The scope of this project is limited to the processing, analysis, and fare prediction of New York City Yellow Taxi trip records for January 2025 using Python and Apache PySpark. The project covers the complete workflow, beginning with the acquisition of raw records and ending with the evaluation of machine-learning models.

The project includes the following components:

- Dataset Acquisition: Retrieval of daily taxi-trip records for January 2025 and organization of the downloaded data into a structured directory.
- Data Conversion and Storage: Conversion of raw JSON Lines data into a typed Parquet dataset to facilitate efficient reading and subsequent processing.
- Data Cleaning: Assessment of data quality and removal of records that violate the project's defined validity criteria, including invalid trip durations, nonpositive distances, nonpositive fares, and extreme trip measurements.
- Feature Engineering: Creation of additional temporal attributes, including pickup hour, day of the week, weekend indicator, and pickup month, to support analytical queries and model development.
- Descriptive Analytics: Use of Spark SQL to investigate hourly and daily trip demand, fare variations across distance categories, frequently used pickup and drop-off zones, passenger counts, and overall dataset statistics.
- Fare Prediction: Development and evaluation of Linear Regression and Random Forest regression models using selected trip attributes, including trip distance, pickup time, day of the week, and pickup and drop-off location identifiers.

- Performance Evaluation: Comparison of the trained models using RMSE, MAE, and (R^2), based on a training and testing split of the prepared dataset.

The implementation is executed locally using PySpark on a Windows-based development environment. Although Apache Spark supports distributed execution, this project does not implement or evaluate a multi-node cluster. Its primary purpose is to demonstrate Spark-based data processing and analytics on a substantial real-world dataset within the available computing environment.

The project is restricted to a single month of Yellow Taxi trip records. It does not include real-time trip ingestion, live fare estimation, deployment of a production prediction service, or analysis of seasonal patterns across multiple years. The fare-prediction models are intended as baseline regression models for academic evaluation rather than as replacements for official taxi-fare calculations.

Future extensions may include processing additional months or years of data, evaluating the workflow on a multi-node Spark cluster, incorporating additional relevant features, and deploying the prediction model through an application interface.

1.5 Overview of the Proposed System

The proposed system is an end-to-end data analytics and fare-prediction pipeline developed using Python and Apache PySpark. It processes New York City Yellow Taxi trip records for January 2025, transforming raw data into a structured dataset suitable for descriptive analysis and machine-learning applications.

The system is organized into a sequence of processing stages, with each stage producing data or results required by the subsequent stage.

1. **Data Acquisition:** The pipeline begins by retrieving daily taxi-trip records from the NYC Open Data platform using its SODA API. The downloaded records are stored as JSON Lines files, organised by date.

2. **Data Conversion:** The raw files are read and converted into a structured dataset with explicitly defined column data types. The resulting data is stored in Parquet format, providing a columnar representation for subsequent processing.

3. **Data Cleaning and Preparation:** The dataset is examined for invalid timestamps, nonpositive trip durations, nonpositive distances and fares, and extreme trip measurements. Records that violate the project's specified cleaning criteria are excluded. A model-ready dataset is subsequently prepared by applying additional fare-related filtering.

4. **Feature Engineering:** Additional attributes are derived from pickup timestamps, including pickup hour, day of the week, weekend indicator, and pickup month. These features support time-based analysis and provide inputs for the prediction models.

5. **Exploratory Data Analysis:** Spark SQL is used to perform aggregations and examine trip demand by hour and day, average fares across distance categories, frequently used pickup and drop-off zones, passenger counts, and overall trip statistics.

6. **Model Training and Evaluation:** Two regression models, Linear Regression and Random Forest, are trained using selected trip attributes. The prepared data is divided into training and testing subsets, and model performance is assessed using RMSE, MAE, and (R^2).

7. **Output and Result Interpretation:** The final stage involves interpreting the analytical results, comparing model performance, and documenting the findings, limitations, and possible future improvements.

The workflow uses Parquet files as intermediate storage between major stages, allowing the processed data to be reused without repeatedly retrieving and converting the original records. Spark DataFrame operations and Spark SQL are used for data transformations and analytical queries, while PySpark's machine-learning functionality supports model preparation, training, and evaluation.

The overall system demonstrates how a substantial transportation dataset can be processed through a structured analytical workflow using Apache PySpark in a local computing environment.

2. PROBLEM ANALYSIS

2.1 Analysis of the Problem

The analysis of large-scale taxi trip data presents several challenges related to data quality, processing efficiency, pattern identification, and predictive modelling. The January 2025 NYC Yellow Taxi dataset used in this project contains approximately 3.47 million records, with a raw data size of around 1.65 GB. Processing this volume of data requires a systematic workflow capable of handling structured records, performing transformations, and supporting analytical queries.

The first challenge is data quality. The raw dataset contains records with missing values, invalid trip durations, non-positive distances and fares, and extreme measurements that may distort statistical summaries and model results. Therefore, the data must be examined and cleaned using defined criteria before meaningful analysis can be performed.

The second challenge is data preparation and processing. The original records are stored in JSON Lines format, which must be converted into a structured and efficient format for repeated processing. Feature engineering is also necessary to derive useful temporal attributes, such as pickup hour, day of the week, weekend indicator, and pickup month.

Another challenge is identifying meaningful patterns in taxi operations. Trip demand, average fares, and trip distances may vary according to pickup time, day of the week, and pickup or drop-off location. These relationships need to be examined through analytical queries rather than assumptions.

Finally, predicting taxi fares requires identifying relationships between the fare amount and available trip attributes. A predictive model must be trained and evaluated using appropriate metrics to assess how closely its predictions match observed fares. The project addresses these challenges through an end-to-end PySpark workflow that combines data cleaning, feature engineering, Spark SQL analysis, and machine-learning models.

2.2 Rationale for the Study

The increasing volume of transportation data creates opportunities to understand travel patterns, fare variations, and demand fluctuations. Analyzing such data can provide useful insights into taxi operations while demonstrating how large datasets can be transformed into meaningful information through data engineering and analytics techniques.

Apache Spark was selected for this project because it provides a unified framework for distributed data processing, SQL-based analytics, and machine learning. Its Python interface, PySpark, enables data manipulation through DataFrames, execution of analytical queries through Spark SQL, and model development using MLlib. These capabilities allow multiple stages of the project to be implemented within a consistent processing environment.

The NYC Yellow Taxi dataset was selected because it contains several relevant attributes, including pickup and drop-off timestamps, trip distance, fare amount, passenger count, and location identifiers. These attributes support the investigation of temporal demand patterns, distance-based fare variations, and relationships between trip characteristics and fare amounts.

The project also provides practical experience in building a complete data-processing pipeline. Converting raw records into Parquet, applying data-quality rules, engineering analytical features, and evaluating machine-learning models demonstrates how different stages of a data science workflow can be integrated.

Therefore, the study serves both an analytical and an educational purpose: it explores patterns in real-world taxi trip data while demonstrating the practical application of PySpark, Spark SQL, and machine-learning techniques to a dataset containing millions of records.

2.3 Feasibility Analysis

The feasibility of the proposed project was assessed in terms of technical resources, data availability, software requirements, and implementation practicality.

Technical Feasibility: The project was implemented using Python and PySpark, with Apache Spark providing data processing, Spark SQL supporting analytical queries, and MLlib enabling machine-learning model development. The implementation was carried out on a local Windows environment using a system equipped with an Intel Core i5-12500H processor and 16 GB of RAM. Although the system was suitable for executing the project workflow, processing a large dataset required careful resource management and could involve disk-based storage of intermediate results.

Data Feasibility: The NYC Yellow Taxi trip records for January 2025 were obtained through the NYC Open Data API. The dataset provided the required trip, fare, timestamp, and location attributes for analysis and prediction. Its availability in a machine-readable format made it suitable for automated acquisition, conversion, cleaning, and processing.

Software Feasibility: The project relied on Python, PySpark, Apache Spark, Spark SQL, and MLlib. These tools supported the major project requirements, including data transformation, analytical querying, feature engineering, model training, and evaluation. Parquet was used for intermediate datasets to support efficient repeated processing.

Operational and Time Feasibility: The project was divided into independent stages, including data acquisition, format conversion, cleaning, feature engineering, exploratory analysis, and model development. This modular workflow made implementation and testing more manageable within the academic project timeline.

Limitations: The project was executed on a single local machine rather than a multi-node cluster. Consequently, the implementation demonstrates the use of Spark-based processing on a large dataset but does not measure the performance benefits of distributed execution across multiple machines. Model performance is also limited by the selected dataset period, available features, and chosen modelling methods.

Overall, the project was technically and operationally feasible within the available resources and academic timeframe. It provided a practical environment for applying large-scale data processing and machine-learning techniques while identifying opportunities for future improvements through cluster-based execution and expanded datasets.

3. DATASET DESCRIPTION

3.1 Dataset Source

The dataset used in this project consists of NYC Yellow Taxi trip records published through the NYC Open Data platform. The data was accessed using the Socrata Open Data API (SODA), which supports programmatic retrieval of records for data analysis.

The dataset contains trip-level information recorded by the New York City Yellow Taxi system. Its attributes include pickup and drop-off timestamps, trip distance, fare amount, passenger count, and pickup and drop-off location identifiers. These fields provide the necessary information for examining taxi demand patterns, analysing fare variations, and developing fare prediction models.

For this project, records corresponding to January 2025 were retrieved from the dataset using date-based API queries. The downloaded records were stored locally as JSON Lines files and subsequently converted into Parquet format using PySpark for further processing and analysis.

Dataset details:

- **Dataset name:** NYC Yellow Taxi Trip Records
- **Data source:** NYC Open Data
- **API:** Socrata Open Data API (SODA)
- **Dataset identifier:** irx2-vgm4
- **Dataset URL:** https://data.cityofnewyork.us/resource/irx2-vgm4.json
- **Selected period:** January 2025
- **Raw data format:** JSON Lines (JSONL)
- **Processing format:** Apache Parquet

3.2 Dataset Coverage

The project uses NYC Yellow Taxi trip records covering the period from **1 January 2025 to 31 January 2025**. This one-month period provides a consistent dataset for examining variations in taxi demand, fare amounts, and trip characteristics across different hours and days of the week.

Data was collected through the NYC Open Data SODA API using date-based queries. The collection process was divided into daily files, resulting in 31 JSON Lines files corresponding to the days of January 2025. These files were stored locally before being combined and converted into a typed Parquet dataset for subsequent processing.

The selected period includes weekdays and weekends, allowing comparisons of trip activity across the seven days of the week. The pickup timestamps also support hourly demand analysis and the extraction of temporal features for machine-learning experiments.

The coverage is limited to January 2025 and does not include records from other months or years. Consequently, the findings describe patterns observed during the selected month and should not be interpreted as representative of every season or long-term taxi usage trends in New York City.

3.3 Schema and Dataset Volume

The January 2025 dataset contains **3,475,234 raw records**, with an approximate total raw size of **1.65 GB**. The records were downloaded as 31 daily JSON Lines files and converted into a consolidated Parquet dataset for efficient processing and repeated analytical operations.

The dataset contains trip-level attributes describing journey timestamps, distances, fare amounts, passenger information, and taxi-zone identifiers. These attributes support data cleaning, feature engineering, statistical analysis, and fare prediction.

The principal attributes relevant to this project are described below.

Attribute	Description	Role in the Project
tpep_pickup_datetime	Date and time when the trip started	Temporal analysis and feature extraction
tpep_dropoff_datetime	Date and time when the trip ended	Duration calculation and data validation
passenger_count	Recorded number of passengers	Passenger-count analysis
trip_distance	Recorded trip distance in miles	Distance-based analysis and model input
fare_amount	Base fare amount recorded for the trip	Target variable for fare prediction
total_amount	Total trip amount recorded in the dataset	Supporting descriptive analysis
PULocationID	Taxi-zone identifier for the pickup location	Pickup-location analysis and model input
DOLocationID	Taxi-zone identifier for the drop-off location	Drop-off-location analysis and model input

The dataset also contains additional fields, including payment-related and other trip attributes. The attributes listed above were selected because they are directly relevant to the project's analytical objectives and predictive modelling tasks.

During conversion, appropriate data types were assigned to timestamps, numerical measurements, and location identifiers. The resulting Parquet dataset served as the input for subsequent data-quality assessment and cleaning. Following cleaning and preparation, the dataset was used to derive temporal features and conduct Spark SQL analysis.

The raw record count and file size describe the collected dataset before cleaning. Subsequent processing stages reduced the number of records by removing entries that did not satisfy the project's data-quality criteria.

4. TOOLS AND TECHNOLOGIES

4.1 Python

Python was used as the primary programming language for implementing the taxi trip analytics and fare prediction system. Its straightforward syntax and extensive data-processing ecosystem made it suitable for developing the different stages of the project, from data acquisition and conversion to analysis and model evaluation.

Python scripts were developed to retrieve records from the NYC Open Data API, convert raw JSON Lines files into Parquet format, perform data-quality checks, clean the dataset, engineer additional features, execute analytical queries, and train machine-learning models.

Python also provided the interface for Apache Spark through PySpark, enabling the implementation of distributed data-processing operations, Spark SQL queries, and regression algorithms within a consistent programming environment.

Therefore, Python served as the foundation for integrating the project's data engineering, analytical processing, and predictive modelling components.

4.2 Apache Spark and PySpark

Apache Spark is an open-source data processing engine designed to perform large-scale data processing and analytical operations. It provides capabilities for processing structured datasets, executing SQL queries, and supporting machine-learning workflows. These capabilities make it suitable for projects involving millions of records and multiple stages of data transformation.

PySpark is the Python interface for Apache Spark. It allows developers to use Spark's processing capabilities through Python, including DataFrame operations, data transformations, filtering, aggregations, and integration with Spark SQL and MLlib.

In this project, PySpark was used to process the January 2025 NYC Yellow Taxi dataset containing approximately 3.47 million raw records. It supported the conversion and cleaning of data, calculation of trip duration, creation of temporal features, and preparation of datasets for analysis and model training.

Spark DataFrames were used to represent and transform the trip records, while Spark's execution engine handled the underlying processing operations. This approach provided a consistent framework for working with the dataset throughout the project.

Apache Spark and PySpark therefore formed the core processing layer of the system, connecting data preparation with analytical querying and machine-learning tasks.

4.3 Spark SQL

Spark SQL is a component of Apache Spark that enables structured data processing through SQL queries and DataFrame operations. It provides a convenient way to perform filtering, grouping, aggregation, and statistical analysis on large datasets without requiring the data to be loaded into a conventional relational database.

In this project, Spark SQL was used to analyze the cleaned and feature-engineered NYC Yellow Taxi dataset containing approximately 3.25 million records. The dataset was registered as a temporary view, allowing SQL queries to be executed directly against the trip records.

The analysis focused on several aspects of taxi trip activity:

- **Hourly demand analysis:** Examined the number of trips and average fare and distance for each pickup hour.
- **Day-of-week analysis:** Compared trip volumes and average fares across weekdays and weekends.
- **Fare by distance:** Grouped trips into distance bands to investigate the relationship between trip distance and fare amount.
- **Location analysis:** Identified the most frequently recorded pickup and drop-off taxi-zone identifiers.
- **Passenger analysis:** Examined trip counts, average fares, and distances across recorded passenger-count categories.
- **Overall statistics:** Calculated summary measures such as total trip count, average fare, average distance, and average trip duration.

These queries helped identify patterns in the dataset, including variations in demand throughout the day and increasing average fares across longer-distance trip categories.

Spark SQL therefore served as the primary analytical querying component of the project, enabling structured exploration of millions of trip records within the PySpark processing environment.

4.4 Apache Parquet

Apache Parquet is an open-source, column-oriented file format designed for efficient storage and analytical processing of structured data. It is supported by Apache Spark and allows datasets to be stored with their schema and data types preserved.

In this project, Parquet was used to store the taxi trip records after converting them from the original JSON Lines format. The converted dataset provided a structured input for subsequent data cleaning, feature engineering, Spark SQL analysis, and machine-learning operations.

Parquet was also used to store intermediate datasets generated during the preprocessing pipeline, including the cleaned dataset, model-ready dataset, and feature-engineered dataset. This allowed individual stages of the workflow to read previously processed data without repeatedly downloading and converting the original files.

The use of Parquet helped organize the project's data-processing workflow and provided a consistent storage format for successive stages of analysis. Its compatibility with PySpark made it suitable for managing the large structured dataset used in this project.

4.5 PySpark MLib

PySpark MLlib is the machine-learning component of Apache Spark that provides tools for building and evaluating machine-learning pipelines. It supports data preparation, feature transformation, model training, and evaluation within the Spark ecosystem.

In this project, PySpark MLlib was used to develop two regression models for predicting taxi fare amounts: Linear Regression and Random Forest Regression. The models were trained using selected trip attributes, including trip distance, pickup hour, pickup day of the week, pickup location identifier, and drop-off location identifier.

The categorical pickup and drop-off location identifiers were transformed using StringIndexer and OneHotEncoder. VectorAssembler was then used to combine the selected features into a feature vector suitable for model training. The dataset was divided into training and testing subsets using an 80:20 split with a fixed random seed.

Linear Regression was implemented as a baseline model to estimate the relationship between the selected trip features and fare amount. Random Forest Regression was used as a second approach to model potentially nonlinear relationships through an ensemble of decision trees.

The models were evaluated using three standard regression metrics:

- **Root Mean Squared Error (RMSE**): Measures the square root of the average squared prediction error, giving greater weight to larger errors.
- **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and actual fare amounts.
- **Coefficient of Determination ((R^2)):** Measures how well the model explains variation in the target variable relative to a baseline prediction.

The Linear Regression model achieved an RMSE of 4.9823, an MAE of 2.7641, and an (R^2) score of 0.8991. The Random Forest model achieved an RMSE of 5.0404, an MAE of 2.9266, and an (R^2) score of 0.8968 on the test dataset.

Based on these results, Linear Regression performed slightly better than Random Forest across all three evaluation metrics. These results demonstrate the application of Spark-based machine learning to taxi fare prediction; however, the (R^2) score should not be interpreted as prediction accuracy in percentage terms.

4.6 NYC Open Data and the SODA API

NYC Open Data is a public data platform that provides access to datasets published by New York City agencies and organizations. The platform makes transportation-related records available for research, analysis, and application development.

The Socrata Open Data API (SODA) was used to retrieve the NYC Yellow Taxi trip records required for this project. The selected dataset, identified by the dataset ID irx2-vgm4, was accessed through its API endpoint. Python scripts issued date-based requests to collect records corresponding to January 2025.

The acquisition process divided the collection period into daily files and retrieved records in batches. This approach supported systematic data collection across the entire month and produced 31 JSON Lines files for local processing.

The API provided programmatic access to the source data, reducing the need for manual downloads and allowing the acquisition process to be repeated when required. The retrieved records were subsequently converted into Parquet format for processing with PySpark.

NYC Open Data and the SODA API therefore served as the project's external data source and data acquisition interface, respectively.

4.7 Development Environment and Supporting Libraries

The project was developed and executed in a local Windows environment using Python and Apache Spark. The development system was equipped with an Intel Core i5-12500H processor and 16 GB of RAM. This configuration was used to process the January 2025 taxi trip dataset, perform analytical queries, and train the regression models.

The following tools and libraries supported the implementation:

Tool or Library	Purpose
Python	Primary programming language for implementing the project workflow
PySpark	Data processing, transformations, and integration with Spark
Apache Spark SQL	Querying and analyzing structured trip data
PySpark MLlib	Training and evaluating regression models
Parquet	Storage of converted and processed datasets
NYC Open Data SODA API	Retrieving the source dataset programmatically

The project was organized into separate Python scripts for data acquisition, conversion, quality assessment, cleaning, feature engineering, SQL analysis, and model training. This separation made the workflow easier to execute, test, and maintain.

The implementation was designed for local execution rather than deployment on a multi-node cluster. Although the local environment was sufficient to complete the project, the size of the dataset required attention to memory usage and intermediate data storage during processing and model training.

5. SYSTEM DESIGN AND METHODOLOGY

5.1 Architecture

The project follows a modular data-processing architecture that integrates data acquisition, storage, preprocessing, analytical processing, and machine-learning components. The system was implemented using Python and PySpark in a local Windows environment.

The architecture is organized into the following layers:

1. **Data Acquisition Layer:**
   The source data is retrieved from the NYC Open Data SODA API using Python scripts. The January 2025 records are downloaded in daily batches and stored locally as JSON Lines files.
2. **Data Storage and Conversion Layer:**
   The downloaded files are converted into a structured Parquet dataset with explicitly assigned data types. Separate Parquet datasets are subsequently created for cleaned records, model preparation, and feature engineering. This allows the output of each processing stage to be reused by later stages.
3. **Data Preprocessing Layer:**
   This layer performs data-quality checks, removes records that violate the project's cleaning criteria, calculates trip duration, and prepares the data for analysis. Additional temporal attributes are derived from pickup timestamps, including pickup hour, day of the week, weekend indicator, and pickup month.
4. **Analytical Processing Layer:**
   The prepared dataset is analysed using Spark SQL. Aggregations and queries are used to investigate hourly demand, weekday patterns, fare variation across distance bands, pickup and drop-off activity, and passenger-count distributions.
5. **Machine-Learning Layer:**
   PySpark MLlib is used to train Linear Regression and Random Forest regression models. Selected trip attributes are transformed into numerical feature vectors, and the models are evaluated using a separate test dataset.
6. **Results and Evaluation Layer:**
   The final layer presents the analytical findings and compares the predictive performance of the two models using RMSE, MAE, and (R^2).

The architecture supports two downstream activities: descriptive analysis through Spark SQL and predictive modelling through MLlib. Both use the prepared feature-engineered dataset, while the modelling workflow applies additional input selection and validation.

The system is designed as a reproducible local data-processing workflow rather than a deployed, real-time prediction service.

5.2 Data Pipeline

The data pipeline consists of a series of sequential stages, beginning with data acquisition and ending with the preparation of a feature-engineered dataset. Each stage performs a specific operation and produces an output that can be used by subsequent stages.

Stage 1: Data Acquisition

The January 2025 NYC Yellow Taxi records were retrieved from the NYC Open Data SODA API using the **download_january_taxi.py** script. The records were downloaded in daily batches and stored as 31 JSON Lines files, with a combined raw size of approximately 1.65 GB.

Stage 2: Data Conversion

The **convert_jan_to_parquet.py** script converted the downloaded JSON Lines files into a consolidated Parquet dataset. Explicit data types were assigned to the relevant fields to support consistent processing through PySpark. The resulting dataset was stored as taxi_trips.parquet.

Stage 3: Data Quality Assessment

The dataset was examined to identify invalid and anomalous records. The assessment included checks for missing timestamps, non-positive trip durations, non-positive distances and fares, and unusually large trip measurements. The findings were used to define the cleaning criteria applied in the next stage.

Stage 4: Data Cleaning

The **clean_january_data.py** script calculated trip duration and removed records that did not satisfy the selected validity criteria. These included non-positive trip durations, distances and fares, distances exceeding 100 miles, and trip durations exceeding 180 minutes.

After cleaning, **3,251,063 records** were retained from the original 3,475,234 records, representing approximately 93.55% of the raw dataset.

Stage 5: Model Data Preparation

The **prepare_model_data.py** script prepared a dataset specifically for fare prediction. In addition to the relevant validity checks, the model dataset excluded records with fare amounts greater than $200. This threshold was selected for the project's modelling workflow to limit the influence of extreme fare values; it is a project-specific criterion rather than an official taxi fare rule.

The resulting model-ready dataset contained **3,250,274 records**.

Stage 6: Feature Engineering

The **jan_feature_engineering.py** script derived additional temporal attributes from pickup timestamps. These included pickup hour, day of the week, weekend indicator, and pickup month. The resulting dataset, **taxi_trips_features.parquet**, contained 25 columns and was used for both Spark SQL analysis and model development.

Stage 7: Analytical and Predictive Processing

The prepared feature dataset was used in two downstream workflows. The **jan_sql_analysis.py** script performed analytical queries using Spark SQL, while **jan_train_model.py** and **jan_train_random_forest.py** trained and evaluated the regression models.

This pipeline established a repeatable sequence for processing the raw records and separating data preparation from analysis and prediction. Intermediate Parquet files allowed later stages to reuse prepared data without repeating the earlier conversion and cleaning operations.

5.2 Training Workflow

The training workflow was designed to evaluate the performance of two regression algorithms in predicting taxi fare amounts: Linear Regression and Random Forest Regression. Both models were implemented using PySpark MLlib and trained on the same prepared dataset to enable a fair comparison.

1. Dataset Selection and Target Definition

The feature-engineered Parquet dataset was loaded into PySpark. The fare_amount attribute was selected as the target variable, while trip distance, pickup hour, pickup day of the week, pickup location identifier, and drop-off location identifier were used as input features. Records with invalid values in the selected modelling attributes were excluded.

2. Training and Testing Split

The dataset was divided into training and testing subsets using an 80:20 random split with a fixed random seed of 42. The training subset contained 2,600,660 records, while the testing subset contained 649,614 records. The same split configuration was used for both models to support consistent evaluation.

3. Feature Transformation

Pickup and drop-off location identifiers were treated as categorical features. StringIndexer converted these identifiers into indexed representations, and OneHotEncoder transformed the indexed categories into encoded feature vectors. VectorAssembler then combined the encoded location features with trip distance, pickup hour, and pickup day of the week into a single vector required by the regression algorithms.

4. Model Training

Two regression algorithms were trained:

- Linear Regression: Used as the baseline model, with 50 maximum iterations, a regularization parameter of 0.1, and the elastic net mixing parameter set to 0.
- Random Forest Regression: Used as an alternative model capable of representing nonlinear relationships. The configuration included 20 trees, a maximum depth of 8, a maximum of 64 bins, and a fixed random seed of 42.

The trained models were saved locally so that the fitted models could be reused without repeating the complete training process.

5. Model Evaluation

After training, each model generated fare predictions for the test subset. The predictions were evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination ((R^2)). Lower RMSE and MAE values indicate smaller prediction errors, while a higher (R^2) value indicates a stronger fit relative to a baseline that predicts the mean target value.

The evaluation results were as follows:

Evaluation Metric	Linear Regression	Random Forest
RMSE	4.9823	5.0404
MAE	2.7641	2.9266
R^2	0.8991	0.8968

Linear Regression achieved slightly lower RMSE and MAE values and a slightly higher R^2 score than Random Forest. Therefore, Linear Regression performed marginally better on the selected test dataset. The R^2 score represents the proportion of variation explained by the model under the evaluation setup; it should not be interpreted as a percentage of predictions that are correct.

This workflow provides a consistent basis for comparing a baseline linear model with a tree-based ensemble model on the January 2025 taxi trip dataset.

6. IMPLEMENTATION

6.1 Implementation of the Project

The taxi trip analytics and fare prediction system was implemented using Python and PySpark in a local Windows environment. The implementation was divided into independent scripts, with each script responsible for a specific stage of the workflow. Intermediate datasets were stored in Parquet format to allow subsequent stages to reuse processed data.

The implementation consisted of the following components.

1. **Data Acquisition:**
   The download_january_taxi.py script retrieved January 2025 NYC Yellow Taxi records through the NYC Open Data SODA API. The data was downloaded in daily batches and stored as 31 JSON Lines files, with a combined raw size of approximately 1.65 GB.
2. **Data Conversion and Inspection:**
   The convert_jan_to_parquet.py script converted the downloaded JSON Lines files into a typed Parquet dataset. Additional inspection scripts, including investigate_january_data.py and profile_january_quality.py, were used to examine the dataset structure and investigate data-quality issues.
3. **Data Cleaning:**
   The clean_january_data.py script calculated trip duration and applied the project's data-quality criteria. Records containing invalid timestamps, non-positive durations, non-positive distances or fares, distances exceeding 100 miles, or durations exceeding 180 minutes were excluded. The cleaned dataset contained 3,251,063 records.
4. **Model Data Preparation:**
   The prepare_model_data.py script created a dataset for predictive modelling by applying additional validity checks, including a maximum fare threshold of $200. The resulting model-ready dataset contained 3,250,274 records.
5. **Feature Engineering:**
   The jan_feature_engineering.py script derived temporal attributes from pickup timestamps, including pickup hour, day of the week, weekend indicator, and pickup month. These features were added to the dataset stored as taxi_trips_features.parquet, which contained 25 columns and 3,250,274 records.
6. **Analytical Processing:**
   The jan_sql_analysis.py script loaded the feature-engineered dataset and registered it as a temporary Spark SQL view. Queries were executed to analyse hourly demand, weekday activity, fare variation by distance band, pickup and drop-off locations, passenger counts, and overall trip statistics.
7. **Machine-Learning Implementation:**
   Two scripts were developed for fare prediction. The jan_train_model.py script trained a Linear Regression model, while jan_train_random_forest.py trained a Random Forest regression model. Both implementations used selected trip attributes,

categorical location encoding, an 80:20 training-testing split, and the same evaluation metrics.
The trained models were saved locally under models/january_2025_linear_regression and models/january_2025_random_forest, respectively.

8. **Output Generation and Evaluation:**
The final stage produced analytical summaries and regression evaluation metrics. The Linear Regression model achieved an RMSE of 4.9823, an MAE of 2.7641, and an (R^2) score of 0.8991. The Random Forest model achieved an RMSE of 5.0404, an MAE of 2.9266, and an (R^2) score of 0.8968. These outputs were used to compare the predictive performance of the two approaches.

Overall, the implementation integrated data acquisition, storage, cleaning, feature engineering, analytical querying, and machine learning into a sequential workflow. The use of separate scripts and intermediate Parquet datasets made the processing stages easier to execute and maintain.

7. TESTING AND VALIDATION

7.1 Data Quality Checks

Data quality checks were performed to assess the reliability of the collected taxi trip records before conducting analytical queries and training machine-learning models. The checks focused on identifying invalid timestamps, unrealistic trip measurements, non-positive distances and fares, and extreme values that could affect the analysis.

The initial dataset contained 3,475,234 records. Following data cleaning, 3,251,063 records were retained. This represents a retention rate of approximately 93.55%, with 224,171 records removed during the cleaning stage.

The following checks were applied or investigated during the data-quality assessment:

1. **Timestamp and Trip Duration Validation:**
Pickup and drop-off timestamps were examined for missing or invalid values. Trip duration was calculated from the difference between the drop-off and pickup timestamps. Records with non-positive trip durations or durations exceeding 180 minutes were excluded during cleaning.

2. **Trip Distance Validation:**
Records with non-positive trip distances were removed because they were unsuitable for the intended trip analysis and fare prediction tasks. A maximum distance threshold of 100 miles was also applied to exclude unusually long trips beyond the project's selected limit.

3. **Fare Amount Validation:**
Records with non-positive fare amounts were excluded during cleaning. The data-quality investigation also identified extreme fare values, including a recorded fare of $863,372.12 for a trip distance of approximately 1.6 miles and a duration of approximately 5.4 minutes. This record was considered anomalous and illustrated the importance of examining extreme values before modelling.
For the predictive modelling dataset, an additional maximum fare threshold of $200 was applied. This was a project-specific modelling decision intended to limit the influence of

extreme fare values.

4. **Record Counts and Dataset Consistency:**
   The number of records was checked across the principal preprocessing stages to verify that the expected datasets were generated. The cleaning stage retained 3,251,063 records, while the subsequent model-preparation stage retained 3,250,274 records. The feature-engineered dataset also contained 3,250,274 records, confirming that the feature-engineering stage preserved the model-ready record count.

The results are summarized below.

Validation Stage	Records
Initial raw dataset	3,475,234
After data cleaning	3,251,063
After model data preparation	3,250,274
After feature engineering	3,250,274
Raw-to-clean retention rate	93.55%

The individual quality checks could identify overlapping issues within the same record. Therefore, their individual counts should not be added together to calculate the total number of records removed. The overall record counts at each processing stage provide the appropriate measure of dataset retention.

These checks helped establish a consistent dataset for the subsequent Spark SQL analysis and regression experiments. However, the applied thresholds represent the project's chosen criteria and do not guarantee that every remaining record is free of data-quality issues.

7.2 Model Evaluation Procedure

The predictive performance of the Linear Regression and Random Forest regression models was evaluated using a held-out test dataset. Both models were trained and evaluated using the same data selection criteria, feature set, and train-test split configuration to support a consistent comparison.

1. **Training and Testing Strategy**

   The model-ready feature dataset was divided into training and testing subsets using an 80:20 random split with a fixed random seed of 42. The training subset contained 2,600,660 records, while the testing subset contained 649,614 records. The training data was used to fit the models, and the testing data was reserved for evaluating their predictions on records not used during model fitting.

2. **Evaluation Metrics:**
   Three regression metrics were used to evaluate the models:

- Root Mean Squared Error (RMSE): Calculates the square root of the mean squared difference between predicted and actual fare amounts. Larger prediction errors have a greater influence on this metric.
- Mean Absolute Error (MAE): Calculates the average absolute difference between predicted and actual fare amounts, providing a direct measure of the typical error magnitude.
- Coefficient of Determination ((R^2)): Measures the proportion of variation in the target variable explained by the model relative to a baseline that predicts the mean fare.

Lower RMSE and MAE values indicate smaller prediction errors, whereas a higher (R^2) score generally indicates a better fit under the same evaluation conditions.

3. **Comparison Procedure**:
   Both models were evaluated against the same test subset. Their RMSE, MAE, and (R^2) scores were recorded and compared to identify which approach performed better on the selected dataset.

   Linear Regression achieved an RMSE of 4.9823, an MAE of 2.7641, and an (R^2) score of 0.8991. Random Forest achieved an RMSE of 5.0404, an MAE of 2.9266, and an (R^2) score of 0.8968.

   Linear Regression performed slightly better across all three metrics. The differences were relatively small, so the results indicate a marginal advantage for Linear Regression under the chosen experimental configuration rather than proving that it is universally superior to Random Forest.

The evaluation provides a quantitative basis for comparing the two models. However, the results reflect a random train-test split from a single month of data and should not be interpreted as a guarantee of performance on future months or different operating conditions.

8. RESULTS AND ANALYSIS

8.1 Dataset Retention

Dataset retention was analysed by comparing the number of records available at the beginning of the workflow with the number remaining after cleaning and model data preparation. This comparison provides an overview of the impact of the preprocessing operations on the size of the dataset.

The initial January 2025 dataset contained 3,475,234 records, with an approximate raw storage size of 1.65 GB. Following data cleaning, 3,251,063 records were retained, while 224,171 records were removed because they did not satisfy the project's data-quality criteria.

The retention rate after cleaning was calculated as follows:

Retention Rate = (Records Retained / Initial Records) * 100

= (3,251,063 / 3,475,234) * 100

= 93.55% (approx)

The subsequent model data preparation stage retained 3,250,274 records. This represents a further reduction of 789 records, resulting in an overall retention rate of approximately 93.53% relative to the original dataset.

The results show that approximately 93.5% of the original records were retained for subsequent analysis and modelling. The relatively small reduction during model data preparation indicates that most records remaining after cleaning also satisfied the additional criteria applied to the modelling dataset.

Feature engineering preserved the record count while adding derived temporal attributes. This allowed the same prepared dataset to support both Spark SQL analysis and regression model development.

Overall, the retention results demonstrate that the preprocessing workflow reduced the dataset to meet the selected validity criteria while preserving more than three million trip records for further analysis. However, retention alone does not establish data quality; the effectiveness of cleaning also depends on the suitability of the selected criteria.

8.2 Demand Patterns

The demand analysis was conducted to identify periods of high and low taxi activity and examine variations in trip volume across different days of the week. Spark SQL was used to aggregate trip records by pickup hour and day of the week.

8.2.1 Hourly Demand Analysis

The hourly analysis revealed substantial variation in taxi trip volume throughout the day. Demand was relatively low during the early morning hours and increased as the day progressed.

The highest trip volume occurred at 18:00 (6 PM), with 236,339 recorded trips. In contrast, 04:00 (4 AM) was the quietest hour, with 17,913 trips. The evening peak suggests that taxi activity was particularly high during the early evening period in the selected dataset.

Pickup Hour	Number of Trips	Average Fare ($)	Average Distance (miles)
04:00	17,913	22.00	4.78
08:00	133,676	17.68	2.97
12:00	168,332	17.50	2.94
15:00	204,258	18.65	3.18
17:00	229,110	17.38	2.87
18:00	236,339	16.46	2.69

Table 8.2: Selected hourly demand and trip characteristics

The average fare also varied by pickup hour. The highest hourly average fare was observed at 05:00, at $25.32, alongside an average trip distance of 5.77 miles. At 18:00, when demand was highest, the average fare was $16.46 and the average trip distance was 2.69 miles.

These differences suggest that hourly fare variations were associated, at least in part, with differences in trip distance and the composition of trips. The results do not establish that pickup time alone caused the observed fare differences.

8.2.2 Day-of-Week Analysis

Trip activity also varied across the seven days of the week. Thursday recorded the highest number of trips, at 571,208, followed by Friday with 543,672 and Wednesday with 536,586. Monday recorded the lowest volume, at 343,203 trips.

Day	Number of Trips	Average Fare ($)
Sunday	378,728	18.33
Monday	343,203	18.72
Tuesday	425,805	17.49
Wednesday	536,586	17.72
Thursday	571,208	18.07
Friday	543,672	17.97
Saturday	451,072	17.10

Table 8.3: Trip demand and average fare by day of the week

The weekday results show higher trip volumes from Wednesday through Friday, with Thursday recording the largest count. Average fares did not follow exactly the same pattern as trip volume: Monday had the highest average fare at $18.72 despite recording the fewest trips, while Saturday had the lowest average fare at $17.10.

8.2.3 Key Findings

The demand analysis produced three principal observations:

- **Evening peak:** 18:00 was the busiest pickup hour, while 04:00 was the quietest.
- **Variation across the week:** Thursday had the highest trip volume, whereas Monday had the lowest.
- **Demand and fares differ:** Higher trip volume did not necessarily correspond to a higher average fare, highlighting the importance of examining trip characteristics alongside demand.

These findings describe the observed January 2025 dataset. Additional months and years would be required to determine whether the patterns remain consistent across seasons and longer periods.

8.3 Fare–Distance Relationships

The relationship between trip distance and fare amount was examined by grouping taxi trips into distance bands and calculating the number of trips, average distance, and average fare within each group. This analysis was performed to determine how recorded fares varied with the length of a trip.

8.3.1 Fare Analysis by Distance Band

The results show a clear increase in average fare as trip distance increases. Trips shorter than one mile had an average fare of $7.86, whereas trips covering 10 miles or more had an average fare of $63.04.

Distance Band	Number of Trips	Average Distance (miles)	Average Fare ($)
Under 1 mile	765,307	0.67	7.86
1 to under 3 miles	1,622,830	1.75	12.88
3 to under 5 miles	362,521	3.77	21.77
5 to under 10 miles	273,962	7.20	33.18
10 miles or more	225,654	15.80	63.04

Table 8.4: Average fare across trip-distance categories

The largest group consisted of trips between 1 and under 3 miles, containing 1,622,830 records. Trips shorter than one mile formed the second-largest group, with 765,307 records. Together, these categories demonstrate the substantial contribution of shorter trips to the overall dataset.

Average fares increased consistently across the five distance bands. The average fare for trips of 10 miles or more was approximately eight times the average fare for trips shorter than one mile. This indicates a strong descriptive relationship between trip distance and recorded fare amount.

8.3.2 Interpretation of Results

The observed relationship is consistent with trip distance being an important input for fare prediction. Longer trips generally cover more distance and tend to incur higher fares, making distance a useful explanatory variable for the regression models developed in this project.

However, average fare does not increase at a perfectly constant rate across the distance categories. The results reflect aggregated fare amounts and may also be influenced by trip duration, route characteristics, traffic conditions, surcharges, and other factors not fully represented by the distance bands.

The dataset also contained unusually high fare values, which is why a separate fare threshold was applied during model data preparation. Consequently, these findings should be interpreted in the context of the project's preprocessing criteria.

Overall, the fare–distance analysis identified one of the clearest patterns in the dataset: average recorded fare increased substantially across longer trip-distance categories. This finding supports the inclusion of trip distance as a key feature in the fare prediction workflow.

8.4 Location Analysis and Model Comparison

This section examines the most frequently recorded pickup and drop-off locations and compares the predictive performance of the Linear Regression and Random Forest models.

8.4.1 Pickup Location Analysis

The pickup-location analysis identified the taxi-zone identifiers with the highest trip counts. Location identifiers were grouped to calculate the number of trips originating from each pickup zone and the corresponding average fare.

Pickup Zone ID	Number of Trips	Average Fare ($)
161	161,240	15.05
237	157,912	12.18
236	149,664	12.78
132	134,184	61.80
230	118,185	17.42

Table 8.5: Selected high-volume pickup locations

Zone 161 recorded the highest pickup count, with 161,240 trips, followed by zones 237 and 236. Zone 132 also appeared among the five most frequently recorded pickup zones, with an average fare of $61.80, substantially higher than the averages for the other listed zones.

This difference suggests that pickup frequency and average fare are not necessarily directly proportional. However, additional investigation of trip distances and destination distributions would be required to explain the unusually high average fare associated with zone 132.

8.4.2 Drop-off Location Analysis

A similar aggregation was performed for drop-off locations to identify the zones receiving the highest number of recorded trips.

Drop-off Zone ID	Number of Trips	Average Fare ($)
236	155,602	13.18
237	143,993	12.27
161	124,205	14.92
230	99,820	18.49
170	94,441	14.20

Table 8.6: Selected high-volume drop-off locations

Zone 236 recorded the highest drop-off count, with 155,602 trips, followed by zones 237 and 161. The appearance of zones 236, 237, and 161 in both pickup and drop-off rankings indicates that these zones were important locations within the observed trip activity.

The results use TLC taxi-zone identifiers rather than neighbourhood names. The analysis therefore establishes the most frequently recorded zones but does not, by itself, explain the geographical or economic reasons for their activity.

8.4.3 Model Comparison

The project evaluated two regression algorithms for predicting taxi fare amounts: Linear Regression and Random Forest Regression. Both models used the same training and testing split and the same selected input features.

Metric	Linear Regression	Random Forest
RMSE	4.9823	5.0404
MAE	2.7641	2.9266
(R^2)	0.8991	0.8968

Table 8.7: Comparison of fare prediction models

Linear Regression achieved an RMSE of 4.9823, compared with 5.0404 for Random Forest. Its MAE was 2.7641, compared with 2.9266 for Random Forest. Linear Regression also achieved a slightly higher (R^2) score of 0.8991, while Random Forest achieved 0.8968.

These results indicate that Linear Regression performed marginally better across all three metrics under the selected experimental conditions. The differences were small, suggesting that the more complex Random Forest model did not provide an improvement with the chosen features and configuration.

The (R^2) score of 0.8991 indicates that the Linear Regression model explained approximately 89.91% of the variation in the target fare values relative to the mean-prediction baseline on the evaluated test dataset. It does not mean that 89.91% of individual predictions were correct.

8.4.4 Overall Findings

The location analysis identified several taxi zones with high trip volumes, while the model comparison showed that Linear Regression slightly outperformed Random Forest on the selected test dataset. Together with the demand and fare–distance analyses, these findings demonstrate how PySpark can be used to investigate large-scale transportation data and develop baseline fare prediction models.

The results are specific to the January 2025 dataset and the preprocessing and modelling choices used in this project. Further evaluation across additional time periods and alternative feature configurations would be necessary before drawing broader conclusions about model performance or taxi activity patterns.

9. LIMITATIONS AND FUTURE SCOPE

9.1 Local Execution Constraints

The project was implemented and executed in a local Windows environment equipped with an Intel Core i5-12500H processor and 16 GB of RAM. Apache Spark and PySpark were used to process the January 2025 NYC Yellow Taxi dataset, which initially contained approximately 3.47 million records and occupied around 1.65 GB in raw form.

Although the local environment supported the completion of data preprocessing, Spark SQL analysis, and machine-learning experiments, its available memory and processing resources imposed practical constraints. During processing and model training, some intermediate data could not remain entirely in memory and had to be persisted to disk. This increased reliance on disk-based operations and could affect execution time.

The project used Spark's processing capabilities on a single machine rather than across a multi-node cluster. Consequently, the implementation demonstrates Spark-based data processing on a large dataset but does not measure the performance benefits of distributed execution across multiple machines.

The local setup was appropriate for the academic project's scope, allowing the complete workflow to be developed and evaluated without dedicated cluster infrastructure. Future work could involve executing the pipeline on a multi-node Spark cluster or a cloud-based environment to compare execution time, resource utilization, and scalability across different configurations.

9.2 Limited Temporal Coverage

The analysis was restricted to NYC Yellow Taxi trip records from January 2025. Although this period contained more than three million records and supported meaningful analysis of hourly demand, weekday activity, fare variations, and location-based trip volumes, it represented only one month of transportation activity.

Taxi usage can vary across seasons, holidays, weather conditions, special events, and broader changes in travel behaviour. A single-month dataset cannot adequately establish whether the patterns identified in this project persist throughout the year or across multiple years.

Similarly, the fare prediction models were trained and evaluated using records from the same monthly dataset. Their performance therefore does not establish how effectively they would generalize to future months or to periods with different demand patterns and fare distributions.

Future work could extend the dataset to cover multiple months or several years. This would allow seasonal comparisons, longer-term demand analysis, and evaluation of model performance across different time periods. A chronological training and testing strategy could also be introduced to assess how well models trained on earlier records predict fares in later periods.

9.3 Data Quality Limitations and Possible Extensions

The quality of the analytical results depends on the accuracy and completeness of the source records and the suitability of the preprocessing criteria. The dataset contained invalid trip measurements and extreme values, including a recorded fare of $863,372.12 for a trip of approximately 1.6 miles and 5.4 minutes. Such anomalies could distort descriptive statistics and affect model training if left untreated.

The project addressed these issues through data-quality checks and predefined filtering criteria. These included excluding non-positive trip durations, distances, and fares, as well as trips exceeding 100 miles or 180 minutes. An additional maximum fare threshold of $200 was applied during model data preparation.

However, these thresholds were selected for the scope of the project and do not guarantee that all anomalous records were identified or that every retained record was valid. Some unusual trips may be legitimate, while other errors may remain undetected. In addition, missing or inaccurate source attributes may limit the reliability of particular analyses.

The fare prediction models also used a selected set of trip attributes: trip distance, pickup hour, pickup day of the week, pickup location, and drop-off location. Other potentially relevant factors, such as traffic conditions, weather, route characteristics, tolls, and additional fare components, were not comprehensively incorporated into the models. The evaluation results should therefore be interpreted within the limits of the selected features and experimental setup.

Several extensions could improve the project:

- **Enhanced data validation:** Introduce more detailed consistency checks, investigate extreme records individually, and document the effect of alternative filtering thresholds.
- **Expanded datasets:** Include additional months and years to support seasonal analysis and more robust model evaluation.
- **Additional predictive features:** Investigate relevant factors such as trip duration, weather, traffic, and other available trip attributes, while avoiding features that would introduce target leakage.
    - **Model experimentation:** Evaluate alternative hyperparameters and additional regression algorithms to determine whether they provide meaningful improvements over the baseline models.
    - **Stronger evaluation:** Use chronological testing and repeated experiments where appropriate to assess model stability and generalization.
    - **Distributed execution:** Run the pipeline on a multi-node or cloud-based Spark environment to investigate scalability and processing performance.
    - **Operational deployment:** Extend the workflow into a reusable prediction application or dashboard, with appropriate input validation and monitoring.

Overall, the project establishes a functional pipeline for processing, analysing, and modelling large-scale taxi trip data. The limitations identified above provide clear directions for extending the dataset, strengthening validation, improving predictive evaluation, and investigating distributed execution in future work.

12. CONCLUSION

The project, *Scalable Taxi Trip Analytics and Fare Prediction Using Apache PySpark*, focused on developing an end-to-end data-processing and machine-learning workflow using NYC Yellow Taxi trip records for January 2025. The primary objective was to demonstrate how large-scale transportation data could be acquired, processed, analysed, and used to develop predictive models within a consistent Python and PySpark environment.

The project began with the acquisition of 3,475,234 raw trip records through the NYC Open Data SODA API. The records were converted from JSON Lines into Parquet format and processed through a sequence of data-quality checks, cleaning operations, and feature-engineering steps. After cleaning, 3,251,063 records were retained, and the subsequent model-preparation stage produced a dataset containing 3,250,274 records.

Spark SQL was used to investigate taxi demand, fare variations, trip distances, pickup and drop-off locations, and passenger counts. The analysis identified 18:00 as the busiest pickup hour and 04:00 as the quietest. Thursday recorded the highest trip volume among the days of the week, while Monday recorded the lowest. The distance-band analysis also revealed a clear increase in average fare across longer trip categories, with average fares ranging from $7.86 for trips under one mile to $63.04 for trips of 10 miles or more.

For predictive modelling, Linear Regression and Random Forest Regression were implemented using PySpark MLlib. Both models were trained and evaluated using the same general experimental configuration. Linear Regression achieved an RMSE of 4.9823, an MAE of 2.7641, and an (R^2) score of 0.8991. Random Forest achieved an RMSE of 5.0404, an MAE of 2.9266, and an (R^2) score of 0.8968. Under the selected conditions, Linear Regression performed slightly better across all three evaluation metrics.

Overall, the project successfully integrated data acquisition, preprocessing, analytical querying, and regression modelling into a reproducible workflow. It provided practical experience in using PySpark, Spark SQL, Parquet, and MLlib to work with millions of structured records. The project also highlighted the importance of data-quality assessment, appropriate feature selection, and objective model evaluation.

Although the implementation was limited to a single local machine and one month of data, it establishes a foundation for future extensions involving additional time periods, enhanced data validation, alternative features, and distributed execution on a multi-node cluster. The project therefore demonstrates the practical application of large-scale data processing and machine learning to transportation analytics while identifying clear opportunities for further development.